
PROGRAMACIÓN ORIENTADA A OBJETOS

Fundamentos de POO y primeros pasos con Java

PROGRAMACIÓN ORIENTADA A OBJETOS Y ESTRUCTURAS DE DATOS

Carrera de Ingeniería Informática · Facultad de Ciencias Puras

Universidad Autónoma Tomás Frías · Potosí – Bolivia

Stack: [JDK Temurin 25 \(LTS\)](#) · [Visual Studio Code + Extension Pack for Java](#)

00 Presentación de la unidad

Esta unidad introduce el paradigma de la Programación Orientada a Objetos (POO) y sienta las bases de trabajo con el lenguaje Java. Partimos de las ideas conceptuales —qué es un objeto, qué principios organizan este paradigma— y avanzamos hasta escribir, compilar y ejecutar nuestros primeros programas, manejar cadenas y colecciones, y modelar problemas mediante clases, objetos, métodos y constructores. Es la puerta de entrada al resto de la materia: sobre estos cimientos construiremos recursividad, herencia, polimorfismo, estructuras de datos y algoritmos.

› Objetivos de aprendizaje

- Explicar la fundamentación y los principios de la POO, distinguiéndola de la programación estructurada.
- Describir cómo Java compila y ejecuta programas (código fuente → bytecode → JVM) e instalar el entorno de desarrollo.
- Aplicar la sintaxis básica de Java: tipos de datos, variables, operadores y estructuras de control.
- Manipular cadenas de texto y las colecciones fundamentales: listas, mapas (diccionarios) y conjuntos.
- Definir clases, crear objetos y dotarlos de atributos, métodos y constructores.
- Organizar el código con bibliotecas y paquetes, y aplicar una metodología de resolución de problemas con POO.

› Mapa de la unidad

El recorrido se organiza en tres bloques: los fundamentos conceptuales, el lenguaje Java como herramienta, y la construcción de programas orientados a objetos.

FUNDAMENTOS	LENGUAJE JAVA	POO EN JAVA
Fundamentación de la POO	Plataforma y ejecución (JDK/JVM)	Clases y objetos
Principios de la POO	Instalación (Temurin + VS Code)	Métodos · Constructores
Tipos de programas	Sintaxis · Cadenas · Colecciones	Bibliotecas · Resolución de problemas

NOTA

Notación: a lo largo de la guía, los bloques **con fondo oscuro** son fragmentos de código Java; las etiquetas en monoespaciado identifican archivos, comandos o términos técnicos.

01 Fundamentación de la POO

La Programación Orientada a Objetos es un paradigma de programación: una forma de pensar y estructurar el software. Su idea central es representar los elementos de un problema del mundo real como objetos que combinan datos (lo que el objeto sabe) y comportamiento (lo que el objeto puede hacer). En lugar de ver un programa como una secuencia de instrucciones que operan sobre datos separados —visión de la programación estructurada—, la POO lo ve como una colección de objetos que colaboran entre sí enviándose mensajes.

DEFINICIÓN

Paradigma de programación: modelo o enfoque conceptual para diseñar y organizar programas. La POO organiza el software en torno a *objetos* que encapsulan estado y comportamiento.

› ¿Por qué surgió la POO?

A medida que los programas crecieron en tamaño y complejidad, la programación estructurada mostró límites: el código se volvía difícil de mantener, los datos globales eran vulnerables y reutilizar piezas era complicado. La POO responde a estos problemas acercando el modelo del programa al modelo del problema real.

Comparación rápida:

- **Programación estructurada:** el foco está en las *funciones/procedimientos* y en el flujo de control; los datos circulan entre funciones.
- **Programación orientada a objetos:** el foco está en los *objetos*; los datos y las operaciones que los manipulan viven juntos dentro de cada objeto.

› Ventajas principales

- **Modelado natural:** las entidades del problema (estudiante, cuenta, factura) se traducen directamente en clases.
- **Reutilización:** una clase bien diseñada puede usarse en muchos programas.
- **Mantenibilidad:** los cambios quedan localizados dentro de cada clase.
- **Escalabilidad:** facilita construir y organizar sistemas grandes.

02 Principios de la POO

Cuatro principios sostienen el paradigma. En esta unidad los presentamos de forma conceptual; la herencia y el polimorfismo se desarrollan en profundidad en la Unidad 3.

› Abstracción

Consiste en identificar las características esenciales de una entidad e ignorar los detalles irrelevantes para el problema. Al modelar un estudiante para un sistema académico nos interesan su nombre y matrícula, no su color favorito.

› Encapsulamiento

Agrupar los datos y los métodos que operan sobre ellos dentro de una misma unidad (la clase) y ocultar su representación interna, exponiendo solo una interfaz controlada. En Java se logra con modificadores de acceso (`private`, `public`) y métodos de acceso (getters/setters).

> Herencia

Permite crear nuevas clases a partir de clases existentes, reutilizando y extendiendo su comportamiento. Una clase `Docente` puede heredar de una clase `Persona` los atributos comunes y añadir los suyos propios.

> Polimorfismo

Es la capacidad de que un mismo mensaje produzca comportamientos distintos según el objeto que lo recibe. Un método `dibujar()` puede comportarse de forma diferente para un `Círculo` o un `Cuadrado`.

NOTA

En esta unidad usarás abstracción y encapsulamiento constantemente. Herencia y polimorfismo se estudian a fondo más adelante, pero conviene reconocerlos desde ahora como parte del mismo marco.

03 Tipos de programas

Java es un lenguaje de propósito general con el que se pueden construir aplicaciones muy diversas. Conocer los tipos de programas ayuda a ubicar el contexto en el que ejecutaremos nuestro código durante la materia.

- **Aplicaciones de consola:** interactúan por texto (entrada/salida estándar). Serán nuestro punto de partida por su simplicidad.
- **Aplicaciones de escritorio (GUI):** con interfaz gráfica, usando Swing o JavaFX.
- **Aplicaciones web / del lado del servidor:** servicios y sitios web (por ejemplo con Spring).
- **Aplicaciones móviles:** Java es uno de los lenguajes históricos del desarrollo Android.

También podemos clasificar los lenguajes según cómo se ejecutan: compilados (se traducen a código máquina antes de ejecutarse), interpretados (se traducen línea a línea en tiempo de ejecución) e híbridos. Java es un caso híbrido, como veremos a continuación.

04 Plataforma de compilación y ejecución

El modelo de ejecución de Java es una de sus señas de identidad y explica su portabilidad. El proceso tiene tres pasos:

1. Escribimos el **código fuente** en archivos con extensión `.java`.
2. El **compilador** (`javac`) traduce el código fuente a **bytecode**, almacenado en archivos `.class`.
3. La **Máquina Virtual de Java (JVM)** interpreta y ejecuta ese bytecode sobre el sistema operativo concreto.



DEFINICIÓN

WORA — “Write Once, Run Anywhere”: como el bytecode lo ejecuta la JVM y no el sistema operativo directamente, el mismo `.class` corre en cualquier plataforma que tenga una JVM.

› JDK, JRE y JVM

- **JVM (Java Virtual Machine):** ejecuta el bytecode.
- **JRE (Java Runtime Environment):** la JVM más las bibliotecas necesarias para *ejecutar* programas.
- **JDK (Java Development Kit):** el JRE más las herramientas para *desarrollar* (compilador `javac`, depurador, etc.). Para programar necesitamos el JDK.

05 Descarga e instalación del entorno

Trabajaremos con el JDK de Eclipse Temurin en su versión 25, que es una versión de soporte extendido (LTS), y con Visual Studio Code como editor.

› Paso 1 — Instalar el JDK Temurin 25 (LTS)

1. Descargar el instalador de Eclipse Temurin 25 (LTS) para el sistema operativo desde el sitio de Adoptium.
2. Ejecutar el instalador y, si la opción está disponible, marcar “**Set JAVA_HOME variable**” y agregar Java al PATH.
3. Verificar la instalación en una terminal:

TERMINAL

```
java -version
javac -version
```

Ambos comandos deben mostrar la versión 25. Si no se reconocen, revisa que el JDK esté en el PATH.

› Paso 2 — Instalar Visual Studio Code y las extensiones de Java

1. Instalar **Visual Studio Code**.
2. Desde la vista de extensiones, instalar el **Extension Pack for Java** (incluye soporte de lenguaje, depurador, ejecución de pruebas y gestión de proyectos).
3. Crear una carpeta de trabajo, abrirla en VS Code y crear un primer archivo `HolaMundo.java`.

ADVERTENCIA

Java distingue mayúsculas de minúsculas y exige que el archivo se llame igual que la clase pública que contiene. La clase `HolaMundo` debe guardarse como `HolaMundo.java`.

06 Sintaxis de Java

Todo programa Java se organiza dentro de clases. La ejecución comienza en el método `main`. Este es el programa mínimo:

HolaMundo.java

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("¡Hola, mundo!");
    }
}
```

```
}
}
```

Para compilarlo y ejecutarlo desde la terminal:

```

TERMINAL
javac HolaMundo.java // genera HolaMundo.class
java HolaMundo       // ejecuta el programa

```

NOTA

Desde Java 11 también puedes ejecutar un archivo único sin compilarlo aparte con **java HolaMundo.java**. En VS Code basta con pulsar *Run*.

> Tipos de datos primitivos

Java tiene ocho tipos primitivos. Los más usados:

- Enteros: byte, short, int, long (habitualmente int).
- Reales: float, double (habitualmente double).
- Carácter: char (un símbolo entre comillas simples).
- Lógico: boolean (true o false).

NOTA

String no es un tipo primitivo, sino una clase; por eso se escribe con mayúscula inicial.

> Variables, operadores y control

```

Sintaxis.java
int edad = 20;
double promedio = 78.5;
boolean aprobado = true;
char inicial = 'M';
String nombre = "Ana";

// Estructura condicional
if (promedio >= 51) {
    System.out.println("Aprobado");
} else {
    System.out.println("Reprobado");
}

// Bucle
for (int i = 1; i <= 5; i++) {
    System.out.println("Iteración " + i);
}

```

Java ofrece las estructuras de control habituales: condicionales (if, else, switch), bucles (for, while, do-while) y operadores aritméticos, relacionales y lógicos.

> Convenciones de nomenclatura

- **Clases:** PascalCase → Estudiante, CuentaBancaria.
- **Variables y métodos:** camelCase → nombreCompleto, calcularPromedio().
- **Constantes:** MAYÚSCULAS_CON_GUION → PI, NOTA_MINIMA.

07 Cadenas

Una cadena (String) es una secuencia de caracteres. En Java las cadenas son inmutables: una vez creadas no se modifican; las operaciones producen nuevas cadenas. La clase String ofrece numerosos métodos predefinidos.

```
Cadenas.java
String s = "Programación";

int longitud = s.length();           // 12
char primera = s.charAt(0);          // 'P'
String mayus = s.toUpperCase();       // "PROGRAMACIÓN"
String trozo = s.substring(0, 4);     // "Prog"
boolean tiene = s.contains("gram");   // true
String[] partes = "a,b,c".split(","); // ["a", "b", "c"]
```

ADVERTENCIA

Para comparar el contenido de dos cadenas se usa **a.equals(b)**, nunca **a == b**: el operador **==** compara referencias (posiciones en memoria), no el texto.

› StringBuilder: cadenas eficientes

Cuando se construye una cadena por partes (por ejemplo, dentro de un bucle), conviene StringBuilder, que sí es mutable y evita crear muchas cadenas intermedias.

```
StringBuilder.java
StringBuilder sb = new StringBuilder();
sb.append("Hola");
sb.append(", ");
sb.append("mundo");
String resultado = sb.toString(); // "Hola, mundo"
```

08 Listas, diccionarios, conjuntos y tuplas

Para agrupar varios datos, Java ofrece el Java Collections Framework, un conjunto de clases e interfaces del paquete java.util. Los términos “diccionario” y “conjunto” del temario corresponden en Java a los mapas (Map) y a los conjuntos (Set). Todas estas colecciones usan genéricos —los signos < >— para declarar el tipo de los elementos que contienen.

› Listas — List / ArrayList

Una lista es una secuencia ordenada que admite elementos repetidos y acceso por posición (índice).

```
Listas.java
import java.util.ArrayList;
import java.util.List;

List<String> nombres = new ArrayList<>();
nombres.add("Ana");
nombres.add("Luis");

System.out.println(nombres.get(0)); // Ana
System.out.println(nombres.size()); // 2

for (String nombre : nombres) {
    System.out.println(nombre);
}
```

› Diccionarios — Map / HashMap

Un mapa asocia claves con valores (pares clave→valor). Las claves no se repiten. Es la estructura ideal para búsquedas por identificador.

```
Diccionarios.java
import java.util.HashMap;
import java.util.Map;

Map<String, Integer> edades = new HashMap<>();
edades.put("Ana", 20);
edades.put("Luis", 22);

System.out.println(edades.get("Ana"));           // 20
System.out.println(edades.containsKey("Luis")); // true

for (Map.Entry<String, Integer> e : edades.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
```

› Conjuntos — Set / HashSet

Un conjunto almacena elementos sin duplicados y sin un orden garantizado. Es útil cuando solo importa la pertenencia (si un elemento está o no).

```
Conjuntos.java
import java.util.HashSet;
import java.util.Set;

Set<String> etiquetas = new HashSet<>();
etiquetas.add("java");
etiquetas.add("poo");
etiquetas.add("java"); // duplicado: se ignora

System.out.println(etiquetas.size()); // 2
System.out.println(etiquetas.contains("poo")); // true
```

› Tuplas en Java

A diferencia de otros lenguajes, Java no tiene un tipo tupla nativo. Cuando necesitamos agrupar un número fijo de valores relacionados, usamos alternativas:

- **Arreglos**, si los valores son del mismo tipo: `int[] punto = {3, 4};`.
- **Registros (record)**, desde Java 16, para agrupar valores con nombre y tipo —la opción recomendada.
- **Map.Entry**, para pares clave-valor.

```
Tuplas.java
record Punto(int x, int y) {}

Punto p = new Punto(3, 4);
System.out.println(p.x() + ", " + p.y()); // 3, 4
```

NOTA

Regla práctica para elegir: **List** cuando el orden y los repetidos importan; **Map** para buscar por clave; **Set** cuando solo importa la pertenencia sin duplicados.

09 Clases y objetos

DEFINICIÓN

Clase: plantilla o molde que define qué *atributos* (datos) y qué *métodos* (comportamiento) tendrán los objetos de un tipo. **Objeto:** una instancia concreta de una clase, con valores propios en sus atributos.

La clase es el plano; el objeto es la construcción real. De la clase Estudiante podemos crear muchos objetos estudiante, cada uno con su propio nombre y edad.

```
Estudiante.java
public class Estudiante {
    // atributos (estado)
    String nombre;
    int edad;

    // método (comportamiento)
    void saludar() {
        System.out.println("Hola, soy " + nombre);
    }
}
```

Los objetos se crean con el operador new. Cada objeto vive en memoria y se manipula a través de una referencia:

```
Principal.java
Estudiante e1 = new Estudiante();
e1.nombre = "Ana";
e1.edad = 20;
e1.saludar();           // Hola, soy Ana

Estudiante e2 = new Estudiante();
e2.nombre = "Luis";     // e2 es independiente de e1
```

NOTA

Atributos de instancia pertenecen a cada objeto; los **atributos estáticos** (static) pertenecen a la clase y se comparten entre todos los objetos.

10 Métodos predefinidos y definidos por el programador

DEFINICIÓN

Método: bloque de código con nombre que realiza una tarea. Puede recibir *parámetros* y devolver un valor de retorno.

› Métodos predefinidos

Son los que ya vienen en las bibliotecas de Java y podemos invocar directamente.

```
MetodosPredefinidos.java
double raiz = Math.sqrt(16.0);           // 4.0 (biblioteca Math)
System.out.println("Texto");             // biblioteca System
int n = "programa".length();              // 8 (clase String)
```

› Métodos definidos por el programador

Los creamos nosotros. Cada método tiene un tipo de retorno, un nombre, una lista de parámetros y un cuerpo. Java permite la sobrecarga: varios métodos con el mismo nombre pero distinta lista de parámetros.

```
Metodos.java
int sumar(int a, int b) {
    return a + b;
}
```

```
// sobrecarga: mismo nombre, parámetros distintos
double sumar(double a, double b) {
    return a + b;
}

// método sin retorno
void mostrar(String mensaje) {
    System.out.println(mensaje);
}
```

NOTA

Un método **void** no devuelve valor. Un método con tipo de retorno (por ejemplo `int`) debe terminar con **return**.

11 Constructores

DEFINICIÓN

Constructor: método especial que se ejecuta al crear un objeto con `new`. Se llama igual que la clase, no tiene tipo de retorno y sirve para inicializar los atributos.

Si no definimos ningún constructor, Java proporciona uno por defecto (sin parámetros). En cuanto definimos uno propio, ese deja de existir salvo que lo declaremos explícitamente.

Estudiante.java

```
public class Estudiante {
    private String nombre;
    private int edad;

    // constructor parametrizado
    public Estudiante(String nombre, int edad) {
        this.nombre = nombre; // 'this' distingue el atributo del parámetro
        this.edad = edad;
    }

    // constructor por defecto (sobrecarga)
    public Estudiante() {
        this.nombre = "Sin nombre";
        this.edad = 0;
    }
}
```

Principal.java

```
Estudiante a = new Estudiante("Ana", 20); // usa el constructor parametrizado
Estudiante b = new Estudiante();          // usa el constructor por defecto
```

NOTA

La palabra clave **this** hace referencia al objeto actual y se usa para distinguir un atributo de un parámetro con el mismo nombre.

12 Bibliotecas

Java organiza sus clases en paquetes (packages) y ofrece una amplia biblioteca estándar. Para usar una clase de otro paquete la importamos con `import`.

- **java.lang:** se importa automáticamente. Contiene `String`, `Math`, `System`.
- **java.util:** colecciones (`ArrayList`, `HashMap`), `Scanner`, fechas.

- **java.io / java.time**: entrada/salida y manejo de fechas y horas.

```
Paquetes.java
package universidad.registro;    // este archivo pertenece a un paquete

import java.util.ArrayList;      // importamos lo que necesitamos
import java.util.Scanner;
```

Para leer datos del teclado se usa la clase Scanner:

```
Entrada.java
Scanner sc = new Scanner(System.in);
System.out.print("Ingresa tu nombre: ");
String nombre = sc.nextLine();
System.out.println("Hola, " + nombre);
```

NOTA

Además de la biblioteca estándar, existen **bibliotecas externas** distribuidas como archivos .jar y gestionadas con herramientas como Maven o Gradle. Las veremos más adelante en la materia.

13 Resolución de problemas con POO

Resolver un problema con POO es, ante todo, modelarlo: descubrir qué objetos intervienen y cómo colaboran. Una metodología sencilla y efectiva:

1. **Comprender el problema**: ¿qué datos entran, qué resultados se esperan, qué restricciones hay?
2. **Identificar entidades**: los sustantivos suelen ser *clases*; sus características, *atributos*; sus acciones, *métodos*.
3. **Diseñar**: esbozar las clases y sus relaciones (por ejemplo, un diagrama de clases).
4. **Implementar**: traducir el diseño a código Java.
5. **Probar y refinar**: ejecutar con casos de prueba y ajustar.

► Ejemplo integrador — Catálogo de una biblioteca

Reunimos casi todo lo de la unidad: una clase con atributos privados (encapsulamiento), un constructor, un método propio, y una lista de objetos que recorreremos e imprimimos.

```
Libro.java
public class Libro {
    private String titulo;
    private String autor;
    private int anio;

    public Libro(String titulo, String autor, int anio) {
        this.titulo = titulo;
        this.autor = autor;
        this.anio = anio;
    }

    public String ficha() {
        return titulo + " - " + autor + " (" + anio + ")";
    }
}
```

```
Biblioteca.java
import java.util.ArrayList;
import java.util.List;
```

```
public class Biblioteca {
    public static void main(String[] args) {
        List<Libro> catalogo = new ArrayList<>();
        catalogo.add(new Libro("Java a Fondo", "Sznajdleder", 2019));
        catalogo.add(new Libro("Piensa en Java", "Eckel", 2007));

        for (Libro libro : catalogo) {
            System.out.println(libro.ficha());
        }
    }
}
```

EJEMPLO**Salida del programa:****SALIDA**

```
Java a Fondo – Sznajdleder (2019)
Piensa en Java – Eckel (2007)
```

14 Síntesis

- La POO modela el software como objetos que combinan estado (atributos) y comportamiento (métodos), acercando el programa al problema real.
- Sus cuatro principios son abstracción, encapsulamiento, herencia y polimorfismo.
- Java compila el código fuente a bytecode, que la JVM ejecuta en cualquier plataforma (WORA). Para desarrollar se necesita el JDK.
- Las cadenas son inmutables; para construir texto de forma eficiente se usa StringBuilder.
- Las colecciones fundamentales son List (listas), Map (diccionarios) y Set (conjuntos); Java no tiene tuplas nativas.
- Una clase es el molde; un objeto, la instancia. Los constructores inicializan los objetos al crearlos con new.

15 Autoevaluación

Responde con tus propias palabras y comprueba con ejemplos en código cuando sea posible.

1. ¿Qué diferencia fundamental hay entre la programación estructurada y la POO?
2. Explica el flujo desde un archivo .java hasta su ejecución. ¿Qué papel cumple la JVM?
3. ¿Por qué se recomienda comparar cadenas con equals() y no con ==?
4. Indica qué colección usarías para: (a) una agenda telefónica por nombre; (b) la lista de asistentes en orden de llegada; (c) el conjunto de lenguajes distintos que conoce un grupo.
5. ¿Qué diferencia hay entre un atributo de instancia y uno estático?
6. Escribe una clase CuentaBancaria con atributos privados, un constructor y métodos depositar() y retirar().

16 Bibliografía

- Deitel, P. y Deitel, H. Cómo programar en Java. Pearson.

- Sznajdleder, P. Java a Fondo: estudio del lenguaje y desarrollo de aplicaciones. Alfaomega.
- Eckel, B. Piensa en Java. Prentice Hall.
- Oracle. The Java Tutorials — docs.oracle.com/javase/tutorial.
- Eclipse Adoptium (Temurin) — adoptium.net.